

PROMPTS DE IMPLEMENTAÇÃO

Família br_* — Codex e Claude Code

Baseado no Roadmap e Auditoria da Fase 0

Como usar estes prompts

Cada prompt é autocontido e inclui o bloco de contexto obrigatório no início. Para usar:

1. Copie o prompt inteiro — contexto + objetivo + restrições.
2. Cole no Codex (tasks) ou no Claude Code (terminal interativo).
3. Execute sempre na ordem dos IDs dentro de cada fase — há dependências explícitas.
4. Após cada prompt, rode: `ruff check` + `make dev-host-upgrade` + teste de instalação.
5. Abra PR separado por ID de prompt. Um prompt = um commit lógico.

Badge	Ferramenta	Quando preferir
Codex / Claude Code	Ambos	Scaffold, features completas, múltiplos arquivos
Codex	Codex	Tarefas isoladas: fixtures XML, tabelas de dados, ruff fixes
Claude Code	Claude Code	Lógica complexa, integrações SOAP/REST, auditoria de código existente

Fase 0 — Auditoria (já concluída)

F0-1 Auditar gov_* e om_account em busca de reuso

Claude Code Auditoria

PROMPT — F0-1

Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom_addons/br_*

Convencoes:

- Python 3.10, ruff.toml, 4 espacos, snake_case

- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota

- XML IDs: br_modulo.nome_snake

- Dependencias neutras: account, accountant, l10n_br - nunca gov_*

- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,

```

documentar arquivo-fonte em comentario inline: # ref:
gov_account_fiscal_year/models/...
- Testes em custom addons/<modulo>/tests/test *.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Objetivo
Auditar custom addons/gov_* e custom addons/om_account_accountant-19.0.1.0.3
para mapear:
1. O que e contextualizacao GOV setorial (nao reutilizar em br_*)
2. O que e conceito neutro reutilizavel (reimplementar limpo em br_*)
3. Quais parsers bancarios/fiscais valem extracao futura

## Entregavel esperado
Arquivo markdown: custom_addons/br_base/docs/phase0_audit.md
Com secoes:
- gov_modules_sectorial: lista com motivo de exclusao
- gov_modules_donor: lista com arquivo-fonte e valor do conceito
- om_account_modules: leitura de acoplamento de cada sub-modulo
- recommended_optional_ports: br_reports, br_bank_import, br_assets, etc.

## Restricoes
- Nao criar nenhum arquivo Python neste prompt
- Nao modificar nenhum modulo existente
- Apenas leitura e documentacao

```

Fase 1a — br_base

BR-BASE-1 Scaffold completo de br_base

[Codex / Claude Code](#) [Scaffold](#)

PROMPT — BR-BASE-1

```

## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom_addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br modulo.nome snake
- Dependencias neutras: account, accountant, l10n_br — nunca gov_*
- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov_account_fiscal_year/models/...
- Testes em custom addons/<modulo>/tests/test *.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Objetivo
Criar o scaffold completo do modulo custom_addons/br_base seguindo
a estrutura padrao dos modulos br_* deste repositorio.

## Estrutura esperada
custom_addons/br_base/
  __init__.py
  __manifest__.py
  models/
    __init__.py
    res_partner.py          # campos br: cnpj_cpf, tipo_pessoa, ie, im
    res_company.py          # campos br: cnpj, inscricao_estadual
    br_municipio.py         # tabela de municipios IBGE (modelo, sem dados)
  security/

```

```

ir.model.access.csv # ACLs para todos os novos modelos
br_base_security.xml # grupos: br_base.group_user, br_base.group_manager
data/
br_estados.xml # 27 UFs com codigo IBGE e sigla
views/
res_partner_views.xml # heranca: adicionar aba 'Dados BR' em res.partner
res_company_views.xml
br_municipio_views.xml
tests/
init .py
test_br_base_scaffold.py # teste minimo: modulo carrega sem erro

## __manifest__.py esperado
name: BR Base
version: 19.0.1.0.0
depends: ['base', 'mail', 'l10n_br']
category: Localization/Brazil
author: Kodoo

## Restricoes
- Nao implementar validacao de CNPJ/CPF ainda (vem no proximo prompt)
- Nao depender de gov_* em hipotese alguma
- Campos novos em res.partner e res.company via inherit, nunca substituaico
- Arquivo br_estados.xml com dados reais das 27 UFs brasileiras

```

BR-BASE-2 Validação CNPJ / CPF

Codex / Claude Code Feature

PROMPT — BR-BASE-2

```

## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br modulo.nome snake
- Dependencias neutras: account, accountant, l10n_br - nunca gov_*
- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov_account_fiscal_year/models/...
- Testes em custom addons/<modulo>/tests/test *.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Pre-requisito
Modulo br_base scaffolded (BR-BASE-1 concluido).

## Objetivo
Implementar validacao e formatacao de CNPJ e CPF em br_base.

## Arquivo alvo
custom addons/br_base/utils/br_document.py (utilitario puro, sem ORM)

## Funcoes esperadas
validate_cnpj(value: str) -> bool
- remove mascara antes de validar
- rejeita sequencias homogeneas (00000000000000, etc.)
- algoritmo oficial de dois digitos verificadores

validate_cpf(value: str) -> bool
- mesma logica para 11 digitos

format_cnpj(value: str) -> str # XX.XXX.XXX/XXXX-XX
format_cpf(value: str) -> str # XXX.XXX.XXX-XX
strip_document(value: str) -> str # remove tudo que nao for digito

```

```

## Integracao com o modelo
Em custom_addons/br_base/models/res_partner.py:
- campo cnpj_cpf: Char, compute=False, store=True
- @api.constrains('cnpj_cpf', 'tipo_pessoa') que chama validate_cnpj ou validate_cpf
- onchange que aplica format_cnpj / format_cpf automaticamente

## Testes obrigatorios
custom_addons/br_base/tests/test_br_document.py:
- test_cnpj_valido: ao menos 5 CNPJs validos reais
- test_cnpj_invalido: digito verificador errado, sequencia homogenea
- test_cpf_valido: ao menos 5 CPFs validos
- test_cpf_invalido
- test_formatacao_cnpj
- test_formatacao_cpf
- test_strip_mascara

## Restricoes
- br_document.py nao deve importar nada do Odoo (ORM-free, testavel standalone)
- Nao usar bibliotecas externas (brazilnum, validate-docbr): implementar do zero

```

BR-BASE-3 Certificado Digital A1/A3

Codex / Claude Code **Feature**

PROMPT — BR-BASE-3

```

## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom_addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br modulo.nome snake
- Dependencias neutras: account, accountant, l10n_br - nunca gov *
- Reimplementar conceitos de gov * sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov_account_fiscal_year/models/...
- Testes em custom_addons/<modulo>/tests/test_*.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Pre-requisito
BR-BASE-1 e BR-BASE-2 concluidos.

## Objetivo
Criar a camada de gerenciamento de Certificado Digital em br_base.

## Modelo ORM
custom_addons/br_base/models/br_certificado.py
name = 'br.certificado'
company_id: Many2one res.company (required)
tipo: Selection [('a1', 'A1 - Arquivo .pfx'), ('a3', 'A3 - Token/Smartcard')]
arquivo_pfx: Binary # apenas para tipo=a1; armazenado criptografado
senha: Char # armazenada com fields.Char(groups='br_base.group_manager')
validade: Date # extraido automaticamente do certificado
titular: Char # CN do certificado
estado: Selection [('ativo', 'Ativo'), ('expirado', 'Expirado'), ('revogado', 'Revogado')]

## Servico de assinatura
custom_addons/br_base/services/br_sign_service.py
class BrSignService:
    def load_cert_a1(self, pfx_bytes: bytes, password: str) -> tuple[cert, key]
    # usa: from cryptography.hazmat.primitives.serialization import pkcs12
    def sign_xml(self, xml_bytes: bytes, cert, key, reference_uri: str) -> bytes
    # usa: from signxml import XMLSigner
    def verify_xml(self, xml_bytes: bytes) -> bool

```

```

## Regra A3
Para A3: o arquivo pfx fica NO HOST (nao no container Docker).
Criar custom_addons/br base/services/br_a3_proxy.py:
- Comunicacao via Unix socket /tmp/br_a3.sock
- Interface identica ao A1: sign_xml(xml bytes, reference_uri) -> bytes
- Documentar que o processo servidor deve rodar no host com o token USB

## Dependencias Python a adicionar em requirements-gov-general.txt
cryptography>=41.0
signxml>=3.2
lxml>=4.9

## Testes obrigatorios
custom_addons/br base/tests/test_br certificado.py:
- test_load_cert_a1: carregar um .pfx de teste (fixture base64 no proprio teste)
- test_sign_xml: assinar um XML minimo e verificar assinatura
- test_cert_expirado: certificado com validade passada gera estado='expirado'

```

BR-BASE-4 API ViaCEP — autocompletar endereço

Codex **Feature**

PROMPT — BR-BASE-4

```

## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom_addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br modulo.nome_snake
- Dependencias neutras: account, accountant, l10n br - nunca gov *
- Reimplementar conceitos de gov * sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov account fiscal year/models/...
- Testes em custom_addons/<modulo>/tests/test_*.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Objetivo
Integrar ViaCEP em res.partner para autocompletar endereco ao digitar CEP.

## Arquivo
custom_addons/br_base/services/br_cep_service.py
def fetch_cep(cep: str) -> dict | None
# GET https://viacep.com.br/ws/{cep}/json/
# retorna dict com: logradouro, bairro, localidade, uf, ibge
# retorna None se CEP nao encontrado (status 200 mas erro:true)
# timeout: 5s; nao explodir se API estiver fora

## Integracao com res.partner
custom_addons/br_base/models/res_partner.py
@api.onchange('zip')
def onchange_cep br(self):
# so dispara se zip tem 8 digitos numericos
# chama br_cep_service.fetch_cep
# preenche: street (logradouro), city, state_id (busca por uf)
# busca br.municipio pelo codigo ibge retornado e preenche municipio_id
# exhibe warning se CEP nao encontrado, sem bloquear o usuario

## Testes
test_fetch_cep_valido: mockar requests.get, verificar campos retornados
test_fetch_cep_invalido: API retorna erro:true -> None
test_fetch_cep_timeout: requests.get levanta Timeout -> None (nao explode)
test_onchange_preenche_cidade: mockar servico, confirmar city preenchida

## Restricoes
- Nao bloquear save se ViaCEP estiver indisponivel

```

- O onchange e apenas sugestao: usuario pode sobrescrever qualquer campo

Fase 1b — br_account

BR-ACC-1 Scaffold br_account + plano de contas NBC TG

[Codex / Claude Code](#) [Scaffold](#)

PROMPT — BR-ACC-1

```
## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br_modulo.nome_snake
- Dependencias neutras: account, accountant, l10n_br - nunca gov_*
- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov account fiscal year/models/...
- Testes em custom addons/<modulo>/tests/test_*.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Objetivo
Criar o scaffold de br_account com plano de contas NBC TG extraido de l10n_br.

## Estrutura esperada
custom addons/br_account/
  __init__.py
  manifest.py
  models/
    __init__.py
    account_move.py          # campos fiscais BR em account.move
    account_move_line.py     # campos fiscais BR em account.move.line
    account_tax.py           # campos BR: tipo imposto, cst, cfop
    res_company.py           # regime tributario, certificado_id
  security/
    ir.model.access.csv
  data/
    br_chart_of_accounts.xml # plano de contas NBC TG
    br_tax_groups.xml        # grupos: ICMS, IPI, PIS, COFINS, ISS, IRPJ, CSLL
  views/
    account_move_views.xml
    res_company_views.xml
  tests/
    __init__.py
    test_br_account_scaffold.py

## manifest.py
depends: ['br_base', 'account', 'accountant', 'l10n_br', 'l10n_br_sales']

## Campos em account.move
regime_tributario: Selection relacionado a res.company
natureza_operacao: Char
finalidade_emissao: Selection [('1','Normal'),('2','Complementar'),
                               ('3','Ajuste'),('4','Devolucao')]

## Campos em account.move.line
cfop: Char(5)
cst_icms: Char(3)
cst_pis: Char(2)
```

```

cst_cofins: Char(2)
ncm: Char(8)

## Plano de Contas
Usar l10n br como referencia. Extrair estrutura de:
  addons/l10n_br/data/account.account.template.csv (se existir)
Recriar como br chart of accounts.xml com contas NBC TG completas:
  1 - Ativo, 2 - Passivo, 3 - PL, 4 - Receita, 5 - Custo, 6 - Despesa
Cada conta com: code, name, account_type (asset_current, liability_current, etc.)

```

BR-ACC-2 Fiscal Year + Journal Lock — reimplementação limpa de gov

Claude Code **Reimplementação**

PROMPT — BR-ACC-2

```

## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom_addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br_modulo.nome_snake
- Dependencias neutras: account, accountant, l10n_br - nunca gov_*
- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov account fiscal year/models/...
- Testes em custom addons/<modulo>/tests/test_*.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Contexto de reimplementacao
Os modulos gov account fiscal year e gov account journal lock_date
implementam esses conceitos, mas dependem de gov base.
Objetivo: reimplementar o nucleo limpo em br_account SEM herdar gov_*.

## Referencias (ler, nao importar)
# ref: gov_account_fiscal_year/models/account_fiscal_year.py
# ref: gov_account_journal_lock_date/models/account_journal.py
# ref: gov_account_lock_date_update/models/account_lock_date_log.py

## O que implementar

### 1. br.fiscal.year (custom_addons/br_account/models/br_fiscal_year.py)
company_id: Many2one res.company
name: Char
date_from: Date
date_to: Date
state: Selection [('draft','Aberto'),('done','Fechado')]
@api.constrains: impedir sobreposicao de periodos na mesma empresa
def get_fiscal_year(self, company_id, date) -> record # busca por data

### 2. Journal Lock Date (custom_addons/br_account/models/br_journal_lock.py)
inherit = 'account.journal'
lock_date_br: Date # lock date especifico do diario (complementa o da empresa)
@api.constrains em account.move: bloquear posting antes do lock_date_br

### 3. Lock Date Log (custom_addons/br_account/models/br_lock_date_log.py)
_name = 'br.lock.date.log'
company_id, user_id, date_old, date_new, reason, timestamp
# chamado automaticamente quando res.company.period_lock_date muda

## Testes
test_fiscal_year_sem_sobreposicao
test_fiscal_year_busca_por_data
test_journal_lock_impede_posting
test_lock_date_log_criado_na_mudanca

```

Fase 1c — br_tax_engine

TAX-1 Motor tributário dual — modelos e seletor temporal

Codex / Claude Code **Core**

PROMPT — TAX-1

```
## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br modulo.nome snake
- Dependencias neutras: account, accountant, l10n_br - nunca gov_*
- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov_account_fiscal_year/models/...
- Testes em custom addons/<modulo>/tests/test *.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Objetivo
Criar br tax engine: o motor tributario dual que suporta simultaneamente
o regime antigo (ICMS/PIS/COFINS/ISS) e o novo (CBS/IBS/IS) da EC 132/2023.
Este e o modulo mais critico da arquitetura. Seguir o design doc.

## Modelos

### br.tax.rule (models/br_tax_rule.py)
name: Char(required)
tax_type: Selection [
    ('icms','ICMS'), ('ipi','IPI'), ('pis','PIS'), ('cofins','COFINS'),
    ('iss','ISS'), ('irpj','IRPJ'), ('csll','CSLL'),
    ('cbs','CBS - Contrib. s/ Bens e Servicos'),
    ('ibs','IBS - Imposto s/ Bens e Servicos'),
    ('is','IS - Imposto Seletivo'),
]
regime_type: Selection [
    ('simples','Simples Nacional'), ('presumido','Lucro Presumido'),
    ('real','Lucro Real'), ('mei','MEI'), ('all','Todos')
]
date_from: Date(required)
date_to: Date # null = vigente indefinidamente
rate: Float # aliquota percentual
active: Boolean(default=True)

@api.constrains('date from','date to','tax type','regime type')
def check_overlap(self): # nao permitir regras sobrepostas para mesmo tipo+regime

@classmethod
def get_active_rules(cls, env, date, regime, tax_types=None) -> recordset
# domain: date_from<=date, (date_to=False OR date_to>=date),
#         regime_type in [regime,'all'], active=True

### br.tax.engine.mixin (models/br_tax_engine_mixin.py)
Mixin para account.move:
engine_date: Date # data de competencia (default: invoice_date)
regime_tributario: Selection (espelhado de res.company)
tax_engine_mode: Selection [
    ('legacy','Regime Antigo'),
```



```

        ('dual', 'Dual - Transicao 2026'),
        ('new', 'Regime Novo CBS/IBS'),
    ]

    def _get_applicable_rules(self) -> dict # retorna regras por tax_type para
    engine_date

## Fixtures de dados
data/br_tax_rules_legacy_2025.xml # ICMS, PIS, COFINS, ISS, IRPJ, CSLL vigentes
data/br_tax_rules_cbs_ibs_2026.xml # CBS 0.9% + IBS 0.1% a partir de 2026-01-01

## Testes obrigatorios
test_regra_vigente_por_data
test_regra_nao_vigente_apos_date_to
test_sem_sobreposicao_mesma_combinacao
test_modulo_dual_retorna_dois_conjuntos_de_regras
test_fixture_2026_carrega_cbs_ibs

```

TAX-2 Tabela CFOP + CST com vigência

Codex **Dados**

PROMPT — TAX-2

```

## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br modulo.nome snake
- Dependencias neutras: account, accountant, l10n_br - nunca gov_*
- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov_account_fiscal_year/models/...
- Testes em custom addons/<modulo>/tests/test *.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Pre-requisito
TAX-1 concluido.

## Objetivo
Criar tabelas de CFOP e CST com vigencia temporal em br_tax engine.
Referencia conceitual (nao importar): l10n_br_fiscal-14.0/l10n_br_fiscal/

## Modelos

### br.cfop (models/br_cfop.py)
code: Char(4) # ex: '5101'
name: Char
tipo: Selection [
    ('1','Entrada Estadual'), ('2','Entrada Interestadual'),
    ('3','Entrada Exterior'), ('5','Saida Estadual'),
    ('6','Saida Interestadual'), ('7','Saida Exterior')
]
aplicacao: Text # descricao da aplicacao fiscal
date_from, date_to: Date # vigencia (CONFAZ pode alterar)

### br.cst.icms, br.cst.pis, br.cst.cofins (models/br_cst.py)
code: Char(2 ou 3)
name: Char
tipo_tributacao: Selection (tributado, isento, nao_tributado, etc.)
date_from, date_to: Date

## Fixtures
data/br_cfop_table.xml # CFOPs mais comuns (5101, 5102, 6101, 6102, 1101, etc.)
data/br_cst_icms.xml # CST ICMS 00 a 90
data/br_cst_pis_cofins.xml # CST PIS/COFINS 01 a 99

```

```
## Restricao
Fixtures devem ser carregadas apenas uma vez via nouupdate="1"
```

Fase 2a — br_nfe

NFE-1 Scaffold br_nfe + modelo br.nfe + ciclo de vida

[Claude Code](#) [Scaffold](#)

PROMPT — NFE-1

```
## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake_case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br modulo.nome_snake
- Dependencias neutras: account, accountant, l10n_br - nunca gov_*
- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov account_fiscal_year/models/...
- Testes em custom addons/<modulo>/tests/test *.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Pre-requisitos
br_base (com certificado digital), br_account, br_tax_engine.

## Objetivo
Criar o scaffold de br_nfe com o modelo central br.nfe e o ciclo de vida
completo do documento fiscal eletronico.

## __manifest__.py
depends: ['br_base', 'br_account', 'br_tax_engine', 'account', 'stock']

## Modelo br.nfe (models/br_nfe.py)
account_move_id: Many2one account.move (ondelete=cascade)
company_id: Many2one res.company
numero: Integer
serie: Char(3)
chave acesso: Char(44)
protocolo: Char
ambiente: Selection [('1','Producao'),('2','Homologacao')]
estado: Selection [
    ('rascunho','Rascunho'), ('validando','Validando'),
    ('enviando','Enviando'), ('autorizado','Autorizado'),
    ('rejeitado','Rejeitado'), ('cancelado','Cancelado'),
    ('inutilizado','Inutilizado'), ('contingencia','Contingencia'),
]
xml_assinado: Text
xml_retorno: Text
danfe_pdf: Binary
contingencia_tipo: Selection [('scan','SCAN'),('svc_an','SVC-AN'),('svc_rs','SVC-RS')]
motivo_cancelamento: Char
data_emissao, data_autorizacao, data_cancelamento: Datetime

## Tabela de endpoints SEFAZ (models/br_nfe_endpoint.py)
_name = 'br.nfe.endpoint'
uf: Char(2)
ambiente: Selection producao/homologacao
```

```

servico: Selection [
    ('NfeAutorizacao','Autorizacao'),
    ('NfeRetAutorizacao','Retorno Autorizacao'),
    ('NfeCancelamento','Cancelamento'),
    ('NfeInutilizacao','Inutilizacao'),
    ('NfeConsultaProtocolo','Consulta'),
    ('NfeStatusServico','Status'),
]
url: Char
data/br_nfe_endpoints.xml # URLs reais por UF (consultar portal NFe.fazenda.gov.br)

## Fluxo de estados (metodos em br.nfe)
action validar() -> chama XSD validation local
action assinar() -> chama br sign service
action enviar() -> SOAP NFeAutorizacaoLote
action consultar() -> SOAP NFeRetAutorizacao (chamado por cron)
action cancelar() -> SOAP NfeCancelamento
action inutilizar()-> SOAP NfeInutilizacao
action contingencia() -> muda estado, registra contingencia_tipo

```

NFE-2 Geração XML NF-e 4.00 + validação XSD

Claude Code **Feature**

PROMPT — NFE-2

```

## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom_addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br modulo.nome_snake
- Dependencias neutras: account, accountant, l10n br — nunca gov *
- Reimplementar conceitos de gov * sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov account fiscal year/models/...
- Testes em custom_addons/<modulo>/tests/test_*.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Pre-requisito
NFE-1 concluido.

## Objetivo
Implementar geracao do XML NF-e 4.00 a partir dos dados de account.move + br.nfe.

## Arquivo principal
custom_addons/br_nfe/services/br_nfe_xml_generator.py
class BrNfeXmlGenerator:
    def generate(self, nfe_record) -> bytes # retorna XML sem assinatura
    def build ide(self, nfe) # identificacao
    def build emit(self, nfe) # emitente
    def build dest(self, nfe) # destinatario
    def build det(self, nfe) # detalhamento de produtos/servicos
    def build total(self, nfe)
    def build transp(self, nfe)
    def build cobr(self, nfe) # cobranca
    def build infAdic(self, nfe)

## Validacao XSD
custom_addons/br_nfe/services/br_nfe_xsd_validator.py
class BrNfeXsdValidator:
    def validate(self, xml_bytes: bytes) -> list[str] # lista de erros, vazia = ok
    # Schema: static/nfe/schemas/nfe_v4.00.xsd (baixar de nfe.fazenda.gov.br)
    # Usar lxml.etree.XMLSchema

## Testes

```

```

test_xml_gerado_valida_xsd: gerar XML de NF-e minima e validar contra XSD
test_campos_obrigatorios_presentes: chave acesso, CNPJ emitente, valores totais
test_xml_invalido_retorna_erros_xsd

## Restricoes
- Nao enviar para SEFAZ neste prompt (apenas geracao + validacao local)
- Usar lxml, nunca xml.etree (sem suporte a namespace adequado)

```

NFE-3 Integração SEFAZ — envio, retorno, contingência

[Claude Code](#) [Integracao](#)

PROMPT — NFE-3

```

## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake_case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br modulo.nome snake
- Dependencias neutras: account, accountant, l10n br — nunca gov_*
- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov_account_fiscal_year/models/...
- Testes em custom addons/<modulo>/tests/test *.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Pre-requisitos
NFE-1, NFE-2 concluidos. Certificado digital A1 funcional.

## Objetivo
Implementar comunicacao real com SEFAZ via SOAP.

## Arquivo
custom_addons/br_nfe/services/br_sefaz_client.py
class BrSefazClient:
    def __init__(self, certificado: BrCertificado, ambiente: str)
    def autorizar_lote(self, xml_signed: bytes, uf: str) -> dict
    def consultar_retorno(self, rec_ibo: str, uf: str) -> dict
    def cancelar(self, chave: str, protocolo: str, motivo: str, uf: str) -> dict
    def inutilizar(self, cnpj, ano, serie, ini, fim, justificativa, uf) -> dict
    def status_servico(self, uf: str) -> dict
    def get_endpoint(self, uf, servico) -> str # busca br.nfe.endpoint no ORM
    def soap_call(self, url, service, xml) -> bytes # zeep com certificado mutuo TLS

## Contingencia automatica
custom_addons/br_nfe/services/br_nfe_contingencia.py
class BrNfeContingencia:
    FALLBACK_ORDER = ['svc an', 'svc rs']
    def detect_sefaz_down(self, uf: str) -> bool
    def activate(self, nfe record) # muda estado para contingencia
    def try_transmission(self, nfe record) -> bool # tenta cada fallback em ordem

## Cron
data/br_nfe_cron.xml
ir.cron: 'BR NF-e: Consultar lotes pendentes' — a cada 5 minutos
ir.cron: 'BR NF-e: Reenviar contingencias' — a cada 30 minutos

## Testes
test_status_servico_homologacao: chamada real (marcada @tagged('sefaz_integration'))
test_contingencia_ativa_quando_timeout: mockar zeep com Timeout
test_fallback_svc_an_quando_sefaz_down

```

Fase 2b — br_simples

SN-1 Scaffold br_simples + Anexos + cálculo DAS

Codex / Claude Code Feature

PROMPT — SN-1

```
## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom_addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br_modulo.nome_snake
- Dependencias neutras: account, accountant, l10n br - nunca gov_*
- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov account fiscal year/models/...
- Testes em custom_addons/<modulo>/tests/test_*.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Pre-requisito
br_tax_engine concluido.

## Objetivo
Implementar o regime Simples Nacional completo em br_simples.

## __manifest__.py
depends: ['br_tax_engine', 'br_account', 'account']

## Modelos

### br.simples.empresa (models/br_simples_empresa.py)
company_id: Many2one res.company (unique)
data_opcao: Date
anexo_principal: Many2one br.simples.anexo
rbt12: Monetary # Receita Bruta dos ultimos 12 meses (atualizado mensalmente)
sublimite_estadual: Monetary # default 3.6M para alguns estados
state: Selection [('ativa','Ativa'),('suspensa','Suspensa'),('excluida','Excluida')]

### br.simples.anexo (models/br_simples_anexo.py)
code: Selection [('I','I'),('II','II'),('III','III'),('IV','IV'),('V','V')]
name: Char
descricao: Text # ex: 'Comercio', 'Industria', 'Servicos com fator R'
date_from, date_to: Date

### br.simples.aliquota (models/br_simples_aliquota.py)
anexo_id: Many2one br.simples.anexo
receita_min, receita_max: Monetary
aliquota_nominal: Float # %
valor_deduzir: Monetary
# formula: aliquota_efetiva = (RBT12 * aliquota_nominal - valor_deduzir) / RBT12

### br.simples.das (models/br_simples_das.py)
company_id, period: Date (mes de competencia)
rbt12: Monetary
receita_mes: Monetary
aliquota_efetiva: Float # calculado
valor_das: Monetary # calculado
fator_r: Float # para Anexos III e V: folha/RBT12
state: Selection rascunho/calculado/pago
account_payment_id: Many2one account.payment

## Fixtures obrigatorias
```

```
data/br_simples_anexos_2024.xml # Anexos I-V com faixas e deducoes (LC 123/2006)
data/br_simples_anexos_2025.xml # mesmas faixas (ajustar se houver portaria)

## Wizard de calculo
wizard/br_simples_calcular_das.py
    Calcula DAS para competencia selecionada:
    1. Busca rbt12 da empresa
    2. Detecta faixa na tabela do anexo
    3. Calcula aliquota efetiva
    4. Para Anexos III/V: aplica Fator R
    5. Gera br.simples.das com resultado

## Testes obrigatorios
test_calculo_das_anexo_i_faixa_2
test_calculo_das_fator_r_anexo_iii_acima_28pct
test_calculo_das_fator_r_anexo_v_abaixo_28pct
test_alerta_limite_4_8m
test_integracao_tax_engine_modulo_dual_2026
```

Fase 3 — br_esocial + br_hr_payroll

ESOC-1 Scaffold br_esocial — eventos de tabela S-1000 a S-1080

[Claude Code](#) [Scaffold](#)

PROMPT — ESOC-1

```
## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br_modulo.nome_snake
- Dependencias neutras: account, accountant, l10n_br - nunca gov_*
- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov account fiscal year/models/...
- Testes em custom_addons/<modulo>/tests/test_*.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Pre-requisito
br_base concluido (certificado digital funcional).

## Objetivo
Criar br_esocial com os eventos de tabela do grupo S-1000.
Estes eventos definem as "dimensoes" que os eventos nao-periodicos e periodicos
referenciam.
Erros aqui propagam para todos os eventos subsequentes.

## __manifest__.py
depends: ['br_base', 'hr', 'hr_contract']

## Modelo base
### br.esocial.evento (models/br_esocial_evento.py)
    _name = 'br.esocial.evento' # abstract-like: cada evento herda
    company_id: Many2one res.company
    tipo: Char # 'S-1000', 'S-2200', etc.
    nrRec: Char # numero do recibo retornado pelo eSocial
    status: Selection [
        ('pendente','Pendente'), ('enviado','Enviado'),
```

```

        ('processado','Processado'), ('erro','Erro'), ('excluido','Excluido')
    ]
    xml_gerado: Text
    xml_retorno: Text
    data_envio: Datetime
    motivo_erro: Text

    def gerar_xml(self) -> bytes # abstrato: cada evento implementa
    def enviar(self)             # POST para API eSocial REST (REST, nao SOAP)
    def consultar(self)
    def excluir(self)

## Eventos de tabela a implementar neste prompt
br.esocial.s1000 # Informacoes do Empregador
br.esocial.s1005 # Tabela de Estabelecimentos
br.esocial.s1010 # Tabela de Rubricas (verbas da folha)
br.esocial.s1020 # Tabela de Lotacoes Tributarias
br.esocial.s1030 # Tabela de Cargos/Empregos Publicos
br.esocial.s1050 # Tabela de Horarios/Turnos
br.esocial.s1070 # Tabela de Processos Adm/Judiciais
br.esocial.s1080 # Tabela de Operadores Portuarios

## Geracao XML
Usar lxml com o leiaute MOS eSocial v2.5 (https://www.gov.br/esocial/schemas)
Salvar XSDs em: custom_addons/br_esocial/static/schemas/
Um arquivo de geracao por evento: services/generators/s1000_generator.py, etc.

## Testes
test_s1000_xml_valida_xsd
test_s1010 rubrica criada e xml correto

```

HR-1 br_hr_payroll — holerite com INSS + IRRF + FGTS

Codex / Claude Code **Feature**

PROMPT — HR-1

```

## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom_addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br_modulo.nome_snake
- Dependencias neutras: account, accountant, l10n_br — nunca gov_*
- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov account fiscal year/models/...
- Testes em custom addons/<modulo>/tests/test_*.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Pre-requisito
br_esocial scaffolded. hr_payroll_community instalado como base.

## Objetivo
Implementar o calculo da folha brasileira sobre hr_payroll_community.

## __manifest__.py
depends: ['br_esocial', 'hr_payroll_community', 'hr_payroll_account_community']

## Tabelas tributarias (modelos com vigencia)

### br.inss.tabela (models/br_inss_tabela.py)
date_from, date_to: Date
linhas: One2many br.inss.tabela.linha
salario_min, salario_max: Monetary

```

```

    aliquota: Float
    def calcular(self, salario_bruto, date) -> float # progressivo: cada faixa ate o
    limite

### br.irrf.tabela (models/br_irrf_tabela.py)
    date from, date to: Date
    linhas: One2many (base calculo min, base calculo_max, aliquota, parcela deduzir)
    deducao_por_dependente: Monetary
    def calcular(self, base, n_dependentes, data_desconto, date) -> float

## Regras de salario (salary.rule) a criar via fixture XML
    SALARIO_BASE, HE50, HE100, ADICIONAL_NOTURNO
    INSS_EMPREGADO, IRRF, ADIANTAMENTO_DESCONTO
    FGTS_EMPREGADO (informativo), FGTS_PATRONAL
    INSS_PATRONAL_20PCT, RAT, TERCEIROS

## Modelo br.fgts.darf (models/br_fgts_darf.py)
    payslip_run_id: Many2one hr.payslip.run
    company_id, period: Date
    valor_fgts: Monetary
    valor_inss: Monetary
    valor_irrf: Monetary
    guia_fgts_pdf: Binary # gerado automaticamente
    darf_pdf: Binary

## Fixtures obrigatorias
    data/br_inss_tabela_2025.xml # tabela progressiva vigente
    data/br_irrf_tabela_2025.xml # tabela IRRF vigente
    data/br_salary_rules.xml # regras de salario padrao

## Testes obrigatorios
    test_inss_progressivo_tres_faixas
    test_irrf_com_dois_dependentes
    test_fgts_8pct_sobre_base_correta
    test_holerite_completo_salario_5000

```

Fase 4 — br_sped

SPED-1 SPED Writer abstrato + EFD Contribuições

[Claude Code](#) [Feature](#)

PROMPT — SPED-1

```

## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom_addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br modulo.nome snake
- Dependencias neutras: account, accountant, l10n_br - nunca gov_*
- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov_account_fiscal_year/models/...
- Testes em custom_addons/<modulo>/tests/test_*.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Pre-requisito
br_account, br_tax_engine, br_nfe concluidos.

```



```

## Objetivo
Criar a infra base de geracao de arquivos SPED e implementar
o primeiro arquivo: EFD Contribuicoes (PIS/COFINS).

## Classe base
custom addons/br sped/utils/br sped writer.py
class BrSpedWriter:
    def __init__(self, company, period from, period to)
    def write_line(self, registro: str, fields: list) -> str
        # formata: |REG|campo1|campo2|\n
    def write_block(self, bloco: str, registros: list[str]) -> str
    def build_file(self) -> bytes # junta blocos, calcula totais de linhas
    def get_hash_md5(self, content: bytes) -> str
    def validate_version(self, obrigacao: str, version: str) -> bool

## EFD Contribuicoes
custom addons/br sped/efd_contrib/br efd_contrib.py
class BrEfdContrib(BrSpedWriter):
    Blocos a implementar (leiaute atual Receita Federal):
    Bloco 0: abertura, identificacao, configuracao
    Bloco A: documentos de servico (NFS-e)
    Bloco C: documentos de mercadoria (NF-e)
    Bloco F: demais documentos e operacoes
    Bloco M: apuracao PIS/COFINS
    Bloco 9: encerramento e controle

    Registros criticos:
    M200/M600: creditos de PIS/COFINS a apurar
    M210/M610: debitos
    M220/M620: ajustes

## Wizard
custom addons/br sped/wizard/br efd_contrib wizard.py
    Selecionar competencia -> gerar arquivo -> download

## Testes
test writer formata linha corretamente
test bloco 0 campos obrigatorios
test efd_contrib gera arquivo_nao_vazio
test_hash_md5 calculado

```

Prompts Transversais — Qualidade e Governança

QA-1 Ruff + cobertura de testes em todos os br_*

Codex **Qualidade**

PROMPT — QA-1

```

## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br_modulo.nome_snake
- Dependencias neutras: account, accountant, l10n_br - nunca gov_*
- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov_account_fiscal_year/models/...
- Testes em custom addons/<modulo>/tests/test *.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

```

```

## Objetivo
Garantir que todos os modulos br_* passam em ruff e tem cobertura
minima de testes antes de abrir PR.

## Tarefa 1: ruff
ruff check custom_addons/br_*
Corrigir todos os erros encontrados.
Nao usar # noqa exceto onde a regra e genuinamente inapropriada para Odoo.

## Tarefa 2: testes de integracao minimos por modulo
Para cada modulo br_* existente, verificar se existe pelo menos:
- 1 teste de instalacao: modulo carrega sem erro
- 1 teste de modelo: CRUD basico no modelo principal
- 1 teste de constraint: violacao gera erro esperado
Se algum modulo nao tiver esses tres testes, cria-los.

## Comando de validacao
./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf \
-d ktest --test-enable \
-i br_base,br_account,br_tax_engine,br_simples \
--stop-after-init
Saida esperada: nenhum FAIL, nenhum ERROR

```

QA-2 Atualizar AGENTS.md com regras br_*

Codex / Claude Code Governança

PROMPT — QA-2

```

## Contexto obrigatorio (inclua em todo prompt antes do objetivo)

Repositorio: Odoo 19, custom addons/br_*
Convencoes:
- Python 3.10, ruff.toml, 4 espacos, snake case
- Prefixo de modelos: br.<dominio>.<entidade> ex: br.simples.aliquota
- XML IDs: br modulo.nome snake
- Dependencias neutras: account, accountant, l10n br - nunca gov_*
- Reimplementar conceitos de gov_* sem herdar: usar como referencia de logica,
  documentar arquivo-fonte em comentario inline: # ref:
gov_account_fiscal_year/models/...
- Testes em custom_addons/<modulo>/tests/test *.py
- Validar com: ./odoo-bin -c deploy/odoo/kodoo.dev-host.local.conf -d ktest
  --test-enable -i <modulo> --stop-after-init

## Objetivo
Adicionar secao 'br_* Development Rules' no AGENTS.md do repositorio.

## Conteudo a adicionar
### br_* Development Rules

Prefix and module boundaries:
- All Brazilian business/fiscal modules use the br_* prefix
- gov_* is public-sector contextualization, not a foundation for br_*
- Never add gov_* to depends[] of any br_* module

GOV concept reuse pattern:
- If a concept already exists in gov_*, reimplement the clean core in br_*
- Use the gov_* file as logic reference, never as a dependency
- Document the source file inline:
  # ref: gov_account_fiscal_year/models/account_fiscal_year.py

Dependency rule for br account:
  depends: ['br_base', 'account', 'accountant', 'l10n br', 'l10n br sales']
  Never: gov_base, base_accounting_kit, dynamic_accounts_report (Option B)

Option B donor rule:

```

- `base_accounting_kit` and `dynamic_accounts_report` are donor stacks only
- Features from Option B must land as optional `br_*` named modules (`br_reports`, `br_bank import`)
- Option B must never appear in `depends[]` of `br_base`, `br_account`, or `br_tax_engine`

Tax engine temporal validity:

- Never hardcode tax rates in Python
- All rates live in `br.tax.rule` with `date_from` / `date_to`
- All changes to rates = new data record, never a code change

Reforma Tributaria rule:

- Never remove legacy tax fields or models before 2033
- Transition is managed by `active=False` on `br.tax.rule`, not by schema changes
- CBS/IBS rates are activated by inserting records with `date_from=2026-01-01`

— Kodoo · `br_*` Implementation Prompts · Fase 0 concluída · Pronto para codificar —